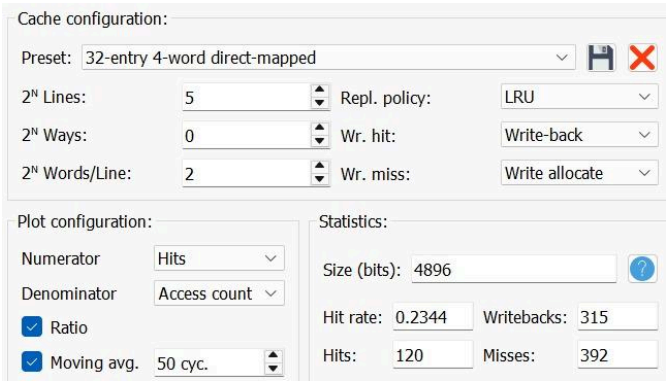
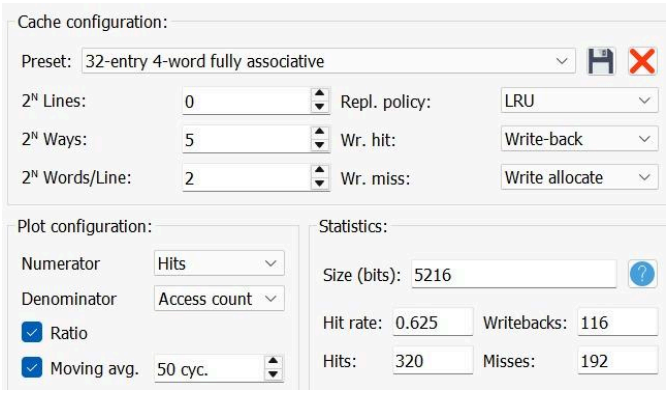
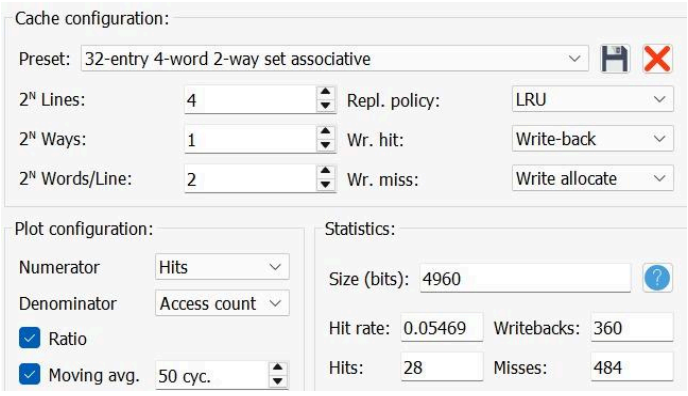
1º preset: 32 entry 4 word direct mapped
lines/number of sets= 2^5=32
ways=2^0=1
words=2^2=4
Data capacity =32*1*4*4=16*32=512
write-hit= write-back
write-miss= write-allocate
Write policy=LRU
hits= 120
misses= 392
write-backs=315
Hit rate= 0,2344= 23,44%



2º 32 entry 4 word fully associative
lines/number of sets= 2^0=1
ways=2^5=32
words=2^2=4
Data capacity = 32*1*4*4=16*32=512
write-hit= write-back
write-miss= write-allocate
Write policy=LRU
hits=320
misses=192
write-backs=116
Hit rate=0,625=62,5%

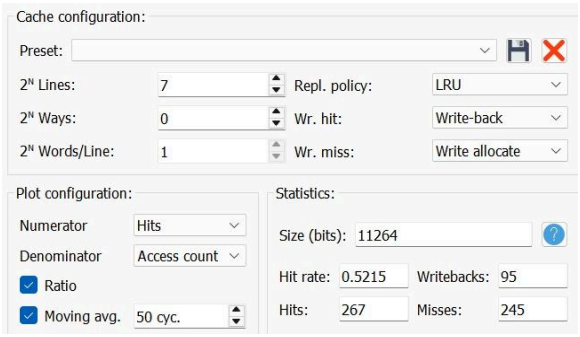


3º 32 entry 4 word 2-way set associative
lines/number of sets= 2^4=16
ways=2^1=2
words=2^2=4
Data capacity = 2*4*16*4=512
write-hit= write-back
write-miss= write-allocate
Write policy=LRU
hits=28
misses=484
write-backs=360
Hit rate=0,05469=5,469%

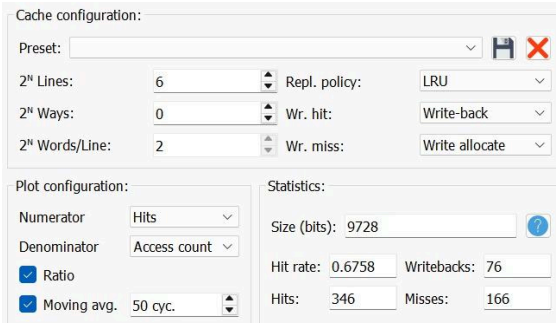


write-hit= write-back
write-miss= write-allocate
Write policy=LRU

1
lines/number of sets= 2^7= 128
ways= 2^0= 1
words= 2^1= 2
Data capacity= 128*1*2*4= 1024
hits= 267
misses= 245
write-backs= 95
Hit rate= 0,5215= 52,15%



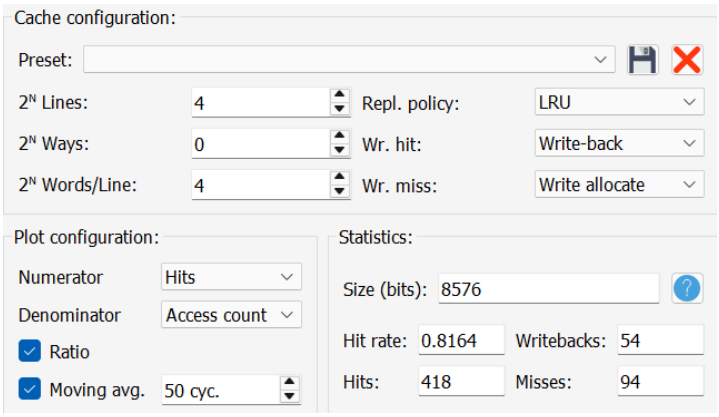
2
lines/number of sets= 2^6= 64
ways= 2^0= 1
words= 2^2= 4
Data capacity= 64*1*4*4= 1024
hits= 346
misses= 166
write-backs= 76
Hit rate= 0,6758= 67,58%



3
lines/number of sets= 2^5= 32
ways= 2^0= 1
words= 2^3= 8
Data capacity= 32*1*8*4= 1024
hits= 414
misses= 98
write-backs= 48
Hit rate= 0.8086= 80,86%

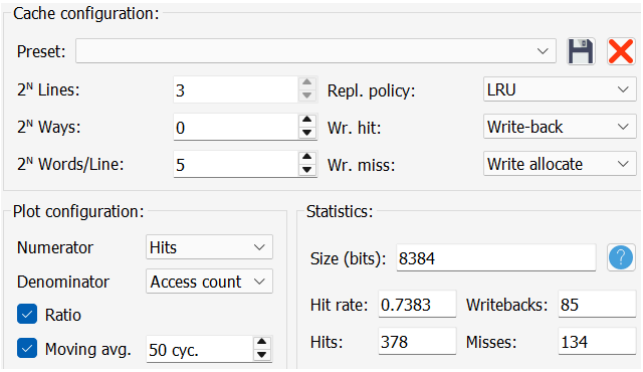


4
lines/number of sets= 2^4= 16
ways= 2^0= 1
words= 2^4= 16
Data capacity = 16*1*16*4= 1024
hits= 418
misses= 94
write-backs= 54
Hit rate= 0,8164= 81,64%



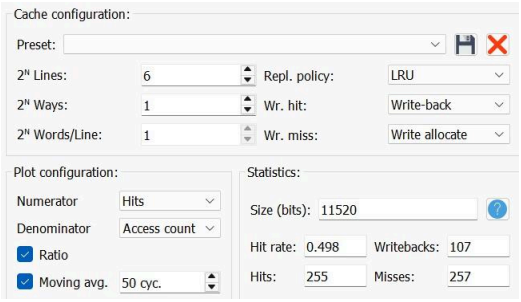
5

lines/number of sets= 2^3= 8
ways= 2^0= 1
words= 2^5= 32
Data capacity = 8*1*32*4= 1024
hits= 378
misses= 134
write-backs= 85
Hit rate= 0,7383= 73,83%



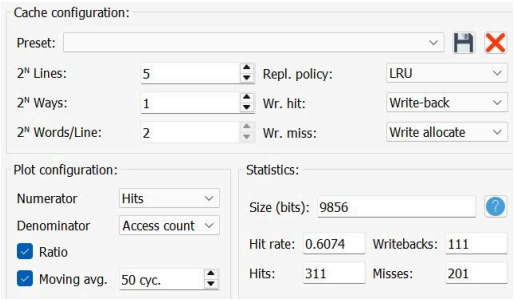
6

lines/number of sets= 2^6= 64
ways= 2^1= 2
words= 2^1= 2
Data capacity = 64*2*2*4= 1024
hits= 255
misses= 257
write-backs= 107
Hit rate= 0.498= 49.8%



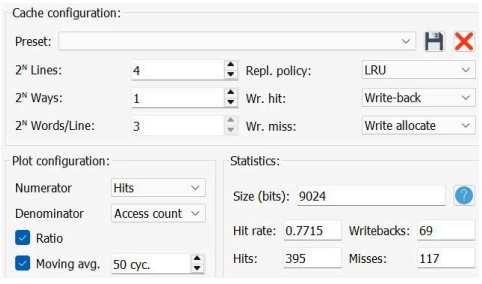
7

lines/number of sets= 2^5= 32
ways= 2^1= 2
words= 2^2= 4
Data capacity= 32*2*4*4= 1024
hits= 311
misses= 201
write-backs= 111
Hit rate= 0,6074= 60,74%

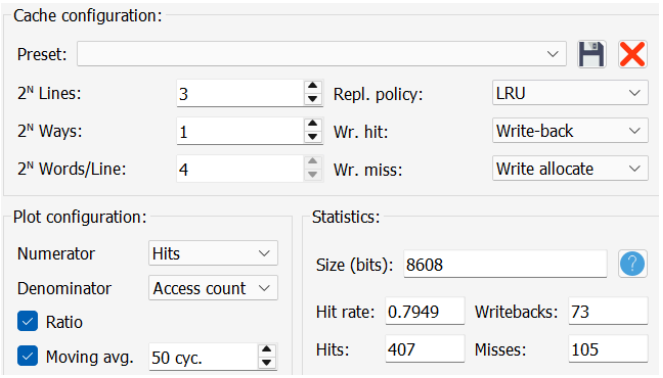


8

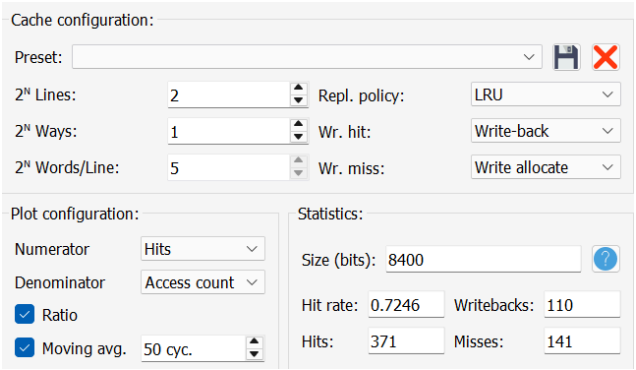
lines/number of sets= 2^4= 16
ways= 2^1= 2
words= 2^3= 8
Data capacity= 16*2*8*4= 1024
hits= 395
misses= 117
write-backs= 69
Hit rate= 0,7715= 77,15%



9
lines/number of sets= 2^3=8
ways= 2^1 =2
words= 2^4 = 16
Data capacity=8*2*16*4 = 1024
hits= 407
misses= 105
write-backs= 73
Hit rate= 0,7949 = 79,49%



10
lines/number of sets= 2^2= 4
ways= 2^1= 2
words= 2^5= 32
Data capacity= 4*2*32*4= 1024
hits= 371
misses= 141
write-backs= 110
Hit rate= 0.7246= 72.46%



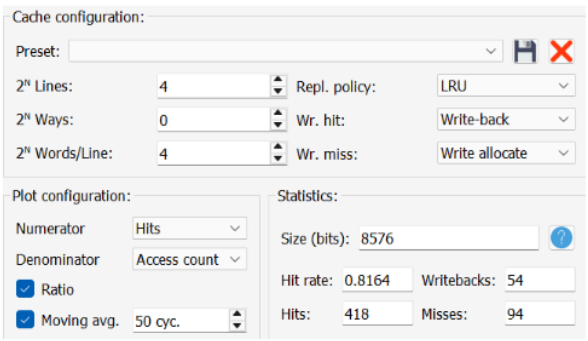
4.3 b) Selecting a configuration

As configurações testadas incluem *direct-mapping* e *2-way set associative* designs, com linhas de 2, 4, 8, 16 e 32 palavras(8 a 128 bytes).

#	S	A	W	Data capacity	Hits	Misses	Writebacks	Hit rate
1	128	1	2	1024 B	267	245	95	52.15%
2	64	1	4	1024 B	346	166	76	67.58%
3	32	1	8	1024 B	414	98	48	80.86%
4	16	1	16	1024 B	418	94	54	81.64%
5	8	1	32	1024 B	378	134	85	73.83%
6	64	2	2	1024 B	255	257	107	49.80%
7	32	2	4	1024 B	311	201	111	60.74%
8	16	2	8	1024 B	395	117	69	77.15%
9	8	2	16	1024 B	407	105	73	79.49%
10	4	2	32	1024 B	371	141	110	72.46%

De todas as configurações a que tem menos *misses* é a configuração 4 com apenas 94.

4
lines/number of sets= 2^4= 16
ways= 2^0= 1
words= 2^4= 16
Data capacity = 16*1*16*4= 1024
hits= 418
misses= 94
write-backs= 54
Hit rate= 0,8164= 81,64%



A configuração escolhida (S = 16, A = 1, W = 16: mapeamento direto, linhas de 64 bytes) teve o menor número de *misses* (94) porque este programa beneficia sobretudo de **localidade espacial**.

Os arrays *a*, *b* e *c* são percorridos sequencialmente, com elementos a 4 bytes de distância. Com linhas de 64 bytes, cada *miss* traz 16 inteiros consecutivos, pelo que cada um destes arrays precisa de cerca de 1 *miss* por cada 16 acessos.

O array *d* é acedido com um salto de 12 bytes, por isso uma linha de 64 bytes ainda serve cerca de 5 dos seus acessos (64 / 12 ≈ 5).

Em contraste, a configuração 1 (W = 2, linhas de 8 bytes) só guarda 2 inteiros por linha, e o salto de 12 bytes de *d* é maior do que a linha. Assim, praticamente todos os acessos a *d* são misses. O resultado é 245 *misses* contra 94.

Nas configurações com mapeamento direto, os *misses* descem de 245 para 166, 98 e 94 quando W passa de 2 para 4, 8 e 16. Em W = 32 a tendência inverte-se (134 misses). As razões prováveis são:

- Com linhas de 128 bytes, a cache de 1 KiB tem só 8 linhas.
- Os arrays *a*, *b* e *c* ocupam 768 bytes e *d* percorre mais ~768 bytes, pelo que o programa trabalha com cerca de 1,5 KiB de endereços, mais do que cabe na cache. Isto é um problema de **capacidade**.
- No mapeamento direto, cada endereço só pode ir para uma linha da cache, que se calcula como "endereço módulo 1024 bytes". A parte final de *d* (endereços acima de 1024 a contar do início de a) cai nas mesmas linhas que *a* e *b* e expulsa-as. Isto é um **conflito**.
- Com poucas linhas muito grandes, estes conflitos pioram, e cada miss traz muitos bytes que não são usados (sobretudo em d).

O simulador não classifica os *misses* por tipo. Esta explicação é por isso uma interpretação do padrão de acessos e não uma medição.

A associatividade não ajudou neste caso. Passar de 1 via para 2 vias, mantendo a capacidade total, reduz para metade o número de sets, e as configurações com 2 vias foram piores do que as equivalentes com mapeamento direto em todos os tamanhos de linha testados (257 vs 245, 201 vs 166, 117 vs 98, 105 vs 94, 141 vs 134).

(Associatividade: com 2 vias, cada endereço pode ir para uma de 2 linhas dentro do seu set, o que ajuda a evitar conflitos. Mas, como a capacidade total é fixa, ter 2 vias significa ter metade dos sets. Aqui não compensou, porque a, b e c já ficam em sets diferentes e quase não entram em conflito entre si. O problema principal é a capacidade, e mais vias não aumentam a capacidade.)

Seria a mesma configuração a melhor para qualquer programa com o mesmo tamanho total de arrays? Não. A melhor configuração depende do padrão de acessos, e não só da quantidade de dados.

- Um programa com saltos grandes ou acessos aleatórios aproveitaria só uma pequena parte de cada linha longa, pelo que linhas mais curtas provavelmente seriam melhores.
- Um programa cujos arrays estejam em endereços que caem nos mesmos sets (por exemplo, separados exatamente pelo tamanho da cache) faria conflito constante numa cache de mapeamento direto, e uma associatividade maior ajudaria muito mais.
- Programas com padrões diferentes de reutilização dos dados (por exemplo, reler repetidamente uma zona pequena) seriam afetados de forma diferente pela capacidade e pela política de substituição.